实验3 进程调度

- 实验目的
  - 撑握进程调度的概念
  - 学习Linux内核源码编写风格，重点理解进程调度策略算法，包括FCFS、RR、SRT、Feedback的调度算法。

实验内容与要求

- 本实验要求在Linux的用户态下编程实现进程调度策略算法的模拟程序
  - 验证test4.c：
    1. FCFS（先到先服务）
    2. RoundRobin（轮循）
  - 新建工程schedule，实现
    3. SRT（最短进程优先）
    4. Feedback（最短剩余时间）

预备知识（1）

- 典型的调度类型：

长程调度、中程调度、短程调度、I/O调度。

- 进程调度是短程调度，执行得最频繁，可以精准地决定下一次执行哪一个进程。

- 进程调度算法包括包括FCFS、RR、SRT、Feedback的调度算法等等。

预备知识（2）

- 调度算法的优劣直接关系到系统的运行效率。
- 一个好的调度算法应该考虑很多方面 :公平、有效、响应时间、周转时间、吞吐量等等。
- 但是这些因素之间又是相互矛盾的 ,最终的取舍根据系统要达到的目标而定

预备知识（3）

进程调度算法：

1.FCFS(先来先服务)：

- 最简单的调度策略，无论任务大小和所需完成时间，对先到的进程先处理，后到的进程后处理。
- 是一种非抢占式策略
- 通常与优先级策略相结合，以提供一种更有效的调度方法
- 特点：非常简单，易于实现；但对于短进程而言，归一化周转时间可能会太大

示例

FCFS调度效果：

```
task id    start  excute
task  1:       0       3
task  2:       2       6
task  3:       4       4
task  4:       6       5
task  5:       8       2
time    : 0                    20
task  1: ###
task  2:     #######
task  3:            ####
task  4:                #####
task  5:                     ##
```

预备知识(4)

2.RR(轮转)：

- 中心思想在于将CPU的运行划分为一个个时间片。如果一个进程在被调度选中之后用完了系统规定的时间片，但未完成要求的任务，则它自行释放自己所占有的CPU而排到就绪队列的末尾，等待下一次调度，同时，进程调度程序又去调度当前就绪队列中的第一个进程。
- 时间片最好略大于一次典型的交互所需时间。
- 是一种抢占式策略。为短进程提供好的响应时间

示例

提示：可创建一个就绪队列，该就绪队列是一个单向循环链表。头结点有个指针变量指向的是正在被调用或将要被调用的进程的进程表项，当进程执行一个时间片后或创建进程时，需将该进程的进程表项添加到就绪队列的尾部

RR调度效果：

```
task id    start  excute
task  1:       0       3
task  2:       2       6
task  3:       4       4
task  4:       6       5
task  5:       8       2
time    : 0                  20
task  1: ## #
task  2:   # # #  #   #   #
task  3:      #  #   #   #
task  4:         #   #   #  ##
task  5:            #   #
```

预备知识(4)

3.SRT（最短剩余时间）

- 在这种情况下，调度程序总是选择预期剩余时间最短的进程。当一个进程加入就绪队列时，它可能比当前运行的进程具有更短的剩余时间，因此只要新进程就绪，调度程序就可能抢占当前正在运行的进程。像SPN一样，调度程序在执行选择函数时必须有关于处理时间的估计，并且存在长进程饥饿的危险。

4.Feedback（反馈）：

- 多级反馈队列算法是轮转算法和优先级算法的综合和发展。设置多个就绪队列，分别赋予不同的优先级，如逐级降低，队列1的优先级最高。每个队列执行时间片的长度也不同，规定优先级越低则时间片越长，如逐级加倍。

操作方法和实验步骤

1.进程表项

```
struct task_struct{
        long pid;        //进程号
        long state;      //进程运行状态
        long priority;   //优先级
        long counter;    //进程剩余时间片
        long start_time; //进程开始时间
        long excute_time; //进程执行时间
};
```

2.进程就绪队列（针对RR）

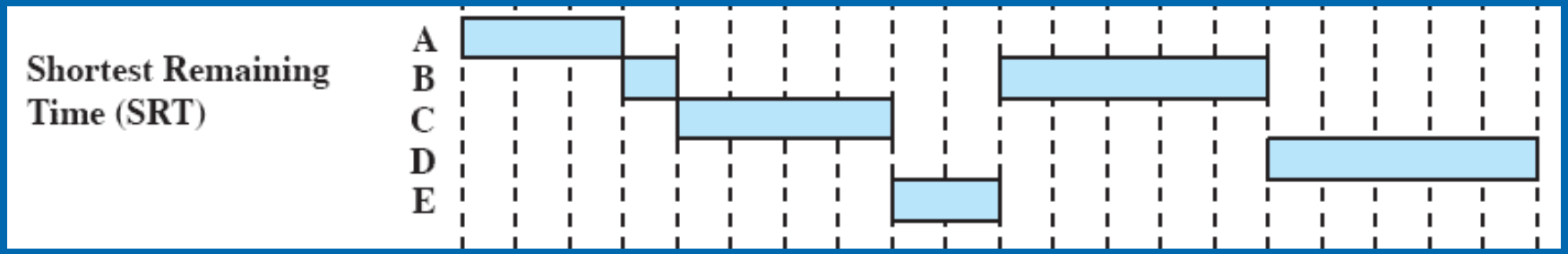
```
struct run_q {struct task_struct *data;    struct run_q *next;};
```

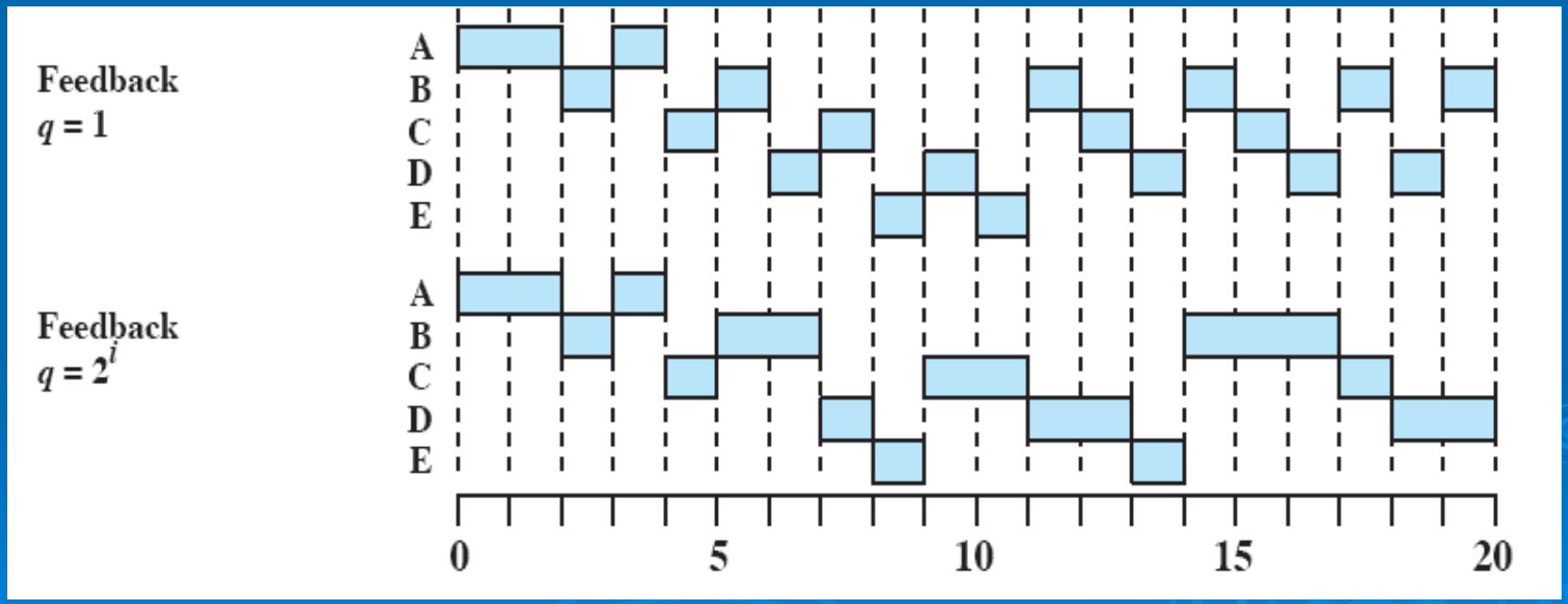
操作方法和实验步骤

3. 将进程定义为全局数组

```
#define  N_PROCESS          5   //进程个数
int process[N_PROCESS][2]={{0,3},{2,6},{4,4},{6,5},{8,2}};//进程初始值
```

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2


Shortest Remaining Time (SRT)
A
B
C
D
E


Feedback
q = 1
A
B
C
D
E
Feedback
q = 2^i
A
B
C
D
E
0
5
10
15
20

操作方法和实验步骤

1. 验证test4.c，理解FIFO和RR两种算法
2. 编写SRT和Feedback两种算法，测试运行结果，编写实验报告。